

eCos Real-Time Operating System

- 
- ▶ Ecos- Embedded Configurable Operating system
 - ▶ The eCos is an open source tool to create embedded kernels and applications

Loading an eCos Configuration

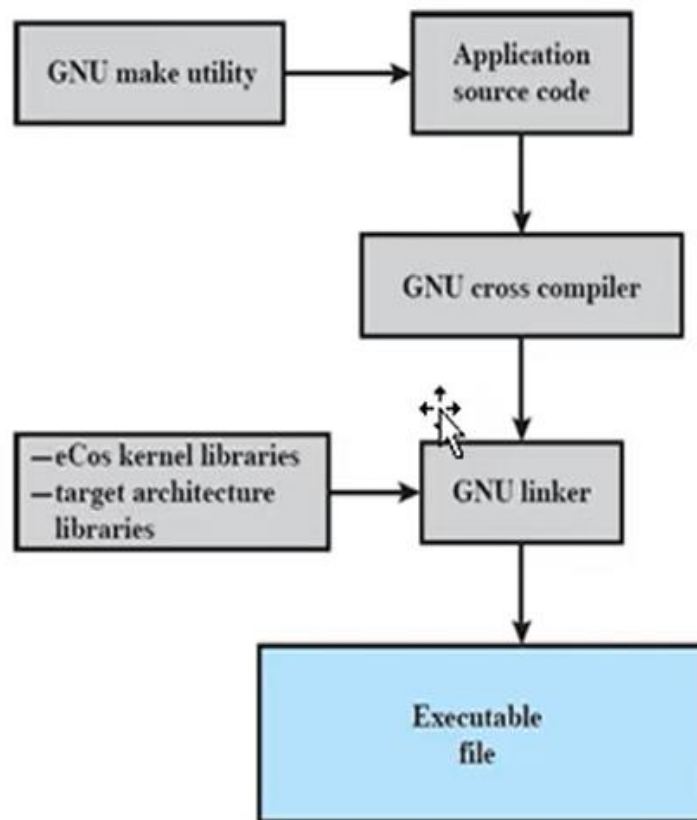
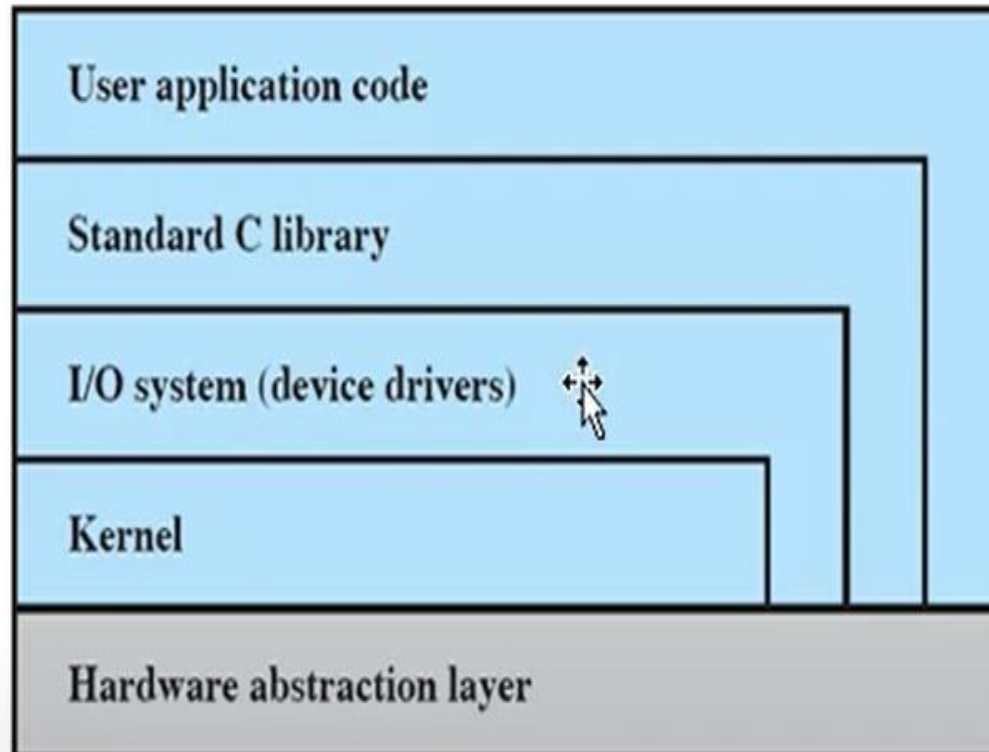
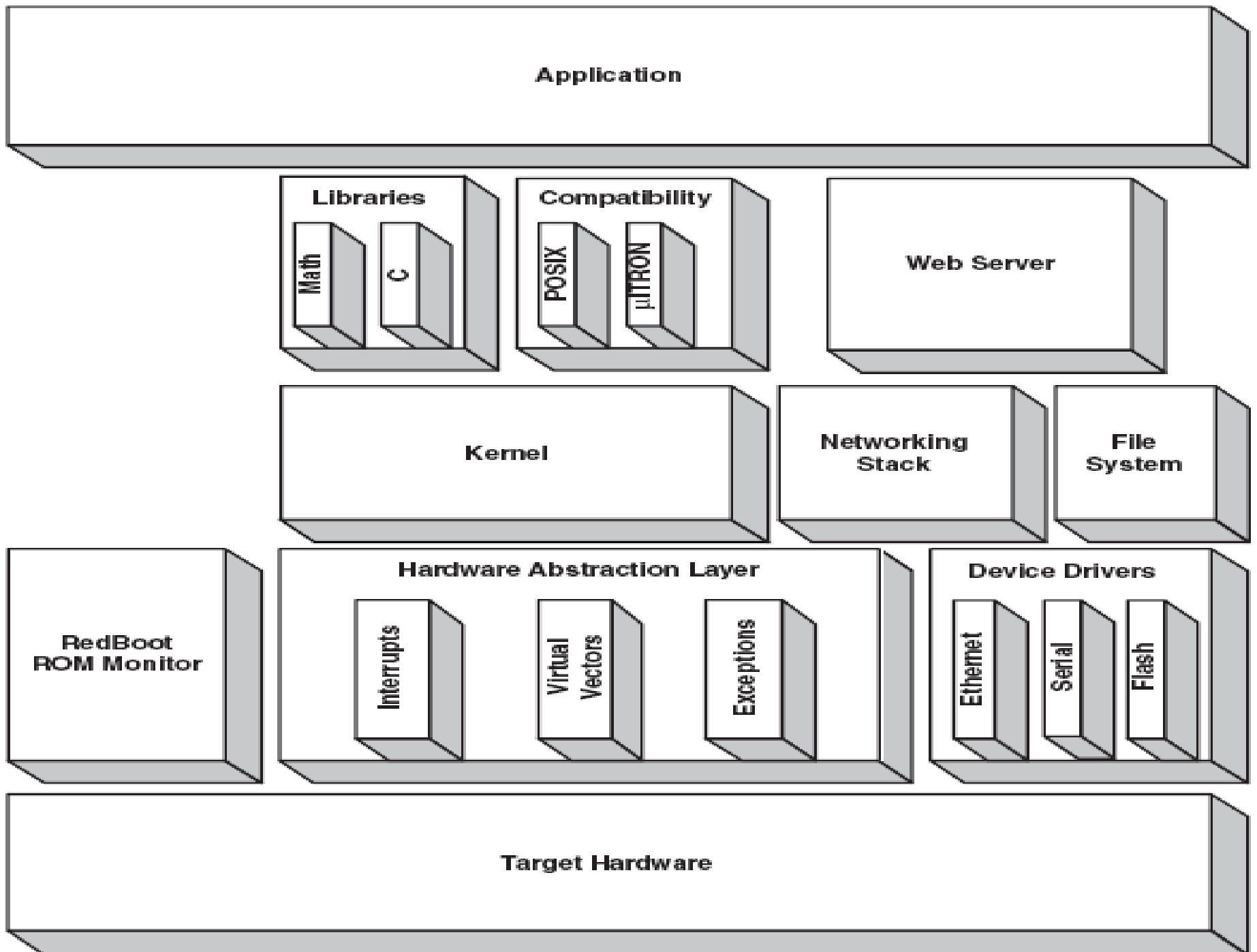


Figure 13.4 Loading an eCos Configuration

eCos

Layered Structure





Configuration System

- ▶ It is the 'heart' of eCos.
 - ▶ Select only the packages that are necessary through configuration.
 - ▶ This reduces the footprint of the application.
-
- eCos uses compile-time control methods.
 - This allows the application writer control over individual lines of code in the packages.
 - The control methods are implemented through C Preprocessor

eCos Components

- ▶ The following are the core components :
 - ▶ Hardware Abstraction Layer
 - ▶ Real-time kernel.
 - ▶ ISO C and math libraries
 - ▶ Device drivers
 - ▶ GNU Debugger (GDB) support
- ▶ The real-time kernel is the central core component.

eCos API

▶ eCos supports the following standard API

- ▶ μ itron
- ▶ POSIX
- ▶ Embedded Linux API compatible with EL/IX.
- ▶ It's own native API.

Hardware Abstraction Layer (HAL)

- ▶ The HAL is a software layer.
- ▶ It provides a platform independent API for platform specific functionality.
- ▶ Enhances portability of code.

Example Scenario

- ▶ Generally on being interrupted, all interrupts are disabled.
- ▶ Bad idea :
 - ▶ Enable interrupts at the end of ISR.
 - ▶ Disadv. : System loses predictability.
- ▶ Good idea :
 - ▶ Enable all interrupts at the start of ISR.
 - ▶ Adv. : interrupts can be pre-empted

The Kernel

- ▶ The Kernel is the core to the eCos system.
- ▶ Provides standard functionality like
 - ▶ interrupt and exception handling
 - ▶ scheduling
 - ▶ threads
 - ▶ synchronization

Kernel API

- ▶ The kernel provides a C API for direct interfacing to the kernel.
- ▶ The kernel API does not return error codes as is usual.
- ▶ Instead it provides a number of assertions that can be enabled or disabled.

Assertions available

- ▶ CYG_FAIL (*diag_message*)
Does not accept a condition as its first argument.
- ▶ CYG_ASSERT (*condition, diag_message*)
Accepts a condition as it's first argument.
- ▶ CYG_ASSERTC (*condition*)
Compact version of the above assertion

Assertions

- ▶ The first two assertions output a diagnostic message that is given as parameter.
- ▶ `CYG_FAIL` outputs the messages irrespective of any conditions.
- ▶ `CYG_ASSERTC()` macro does not output any diagnostic messages.

Exception Handling

- ▶ Exception handling can be done in two ways :
 - ▶ HAL + Kernel Exception Handling
 - ▶ Application Exception Handling
- ▶ HAL + Kernel Exception Handling is the default option.

HAL + Kernel Exception Handling

- ▶ Uses a Vector Service Routine (VSR).
- ▶ It is an array of pointers to exception handler routines.
- ▶ HAL does basic interrupt processing like saving the context etc ...
- ▶ Then control goes to kernel for further processing if required.

Hardware Abstraction Layer

- Presents consistent API to upper layers
- Different for each hardware platform

```
1 #define HAL_ENABLE_INTERRUPTS() \
2     asm volatile ( \
3         "mrs r3, cpsr;" \
4         "bic r3, r3, #0xC0;" \
5         "mrs cpsr, r3;" \
6         : \
7         : \
8         : "r3" \
9         ); \
```

(a) ARM architecture

```
1 #define HAL_ENABLE_INTERRUPTS() \
2     CYG_MACRO_START \
3     cyg_uint32 tmp1, tmp2 \
4     asm volatile ( \
5         "mfmsr    %0;" \
6         "ori      %1,%1,0x800;" \
7         "rlwimi   %0,%1,0,16,16;" \
8         "mtmsr    %0;" \
9         : "=r" (tmp1), "=r" (tmp2)); \
10    CYG_MACRO_END \
```

(b) PowerPC architecture

Figure 13.6 Two Implementations of
HAL_ENABLE_INTERRUPTS () Macro [MASS03]

HAL Modules

- **Architecture:**

- Defines the processor family type.
- This module contains the code necessary for processor startup, interrupt delivery, context switching, and other functionality specific to the instruction set architecture of that processor family.

- **Variant:**

- Supports the features of the specific processor in the family.
- An example of a supported feature is an on-chip module such as a memory management unit (MMU).

eCos Kernel Design

- The eCos kernel was designed to satisfy four main objectives:
 - Low interrupt latency
 - Low task switching latency
 - Small memory footprint
 - Deterministic behavior

eCos I/O System

- Framework for supporting device drivers
- A variety of drivers are available through the configuration package
- Principle objective is efficiency with no unnecessary software layering

Application Exception Handling

- ▶ Applications can provide their own VSR when an exception occurs.
- ▶ VSR's ***must*** be written in assembly language.
- ▶ HAL_VSR_GET and HAL_VSR_SET are macros provided to give access to VSR table.

Interrupt Processing

- ▶ Provides ISR's and DSR's(Deferred **S**ervice **R**outine)
- ▶ ISR's perform most common tasks. They are small and execute quickly.
- ▶ DSR's perform additional processing if necessary.

Interrupt Processing

- ▶ In ISR's calling synchronization primitives is not allowed.
- ▶ They are allowed in DSR.
- ▶ DSR must not make any synchronization calls that block.

Interrupt Processing

- ▶ Synchronization primitives are not allowed inside the ISR's because
 - ▶ ISR's must be fast and bounded by time.
 - ▶ If due to some reason a synchronization primitive causes the task to wait or sleep then it is not acceptable.

Scheduler

- ▶ It's the core of the kernel.
- ▶ eCos provides two schedulers
 - ▶ Multilevel Queue Scheduler
 - ▶ Bitmap Scheduler

Multilevel Queue Scheduler

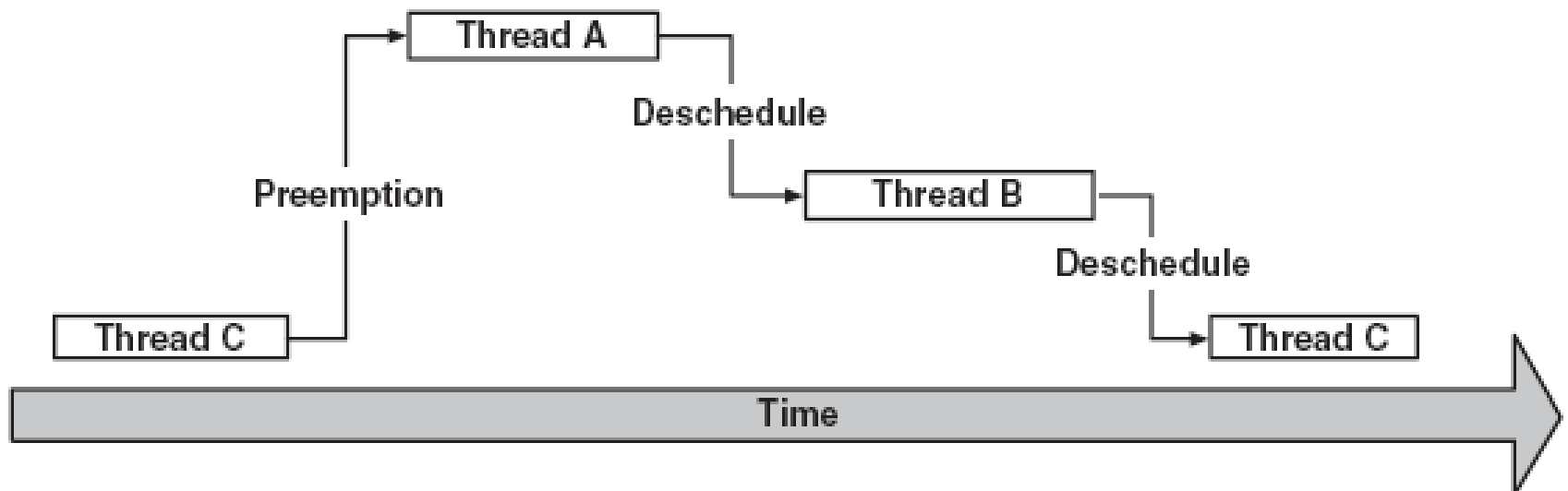
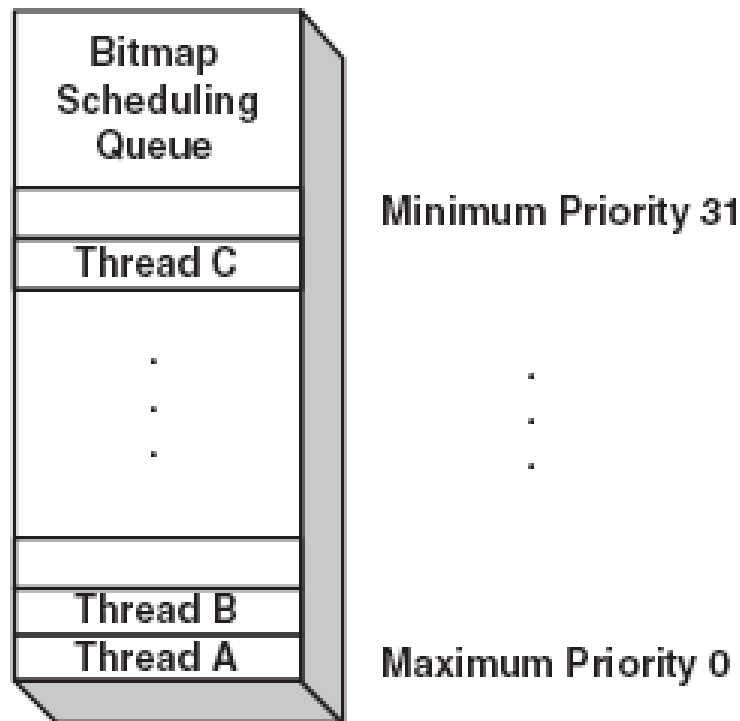
- ▶ Allows multiple threads at same priority level.
- ▶ Allows pre-emption between different priority levels.
- ▶ Timeslicing within a priority level allowed.

Bitmap Scheduler

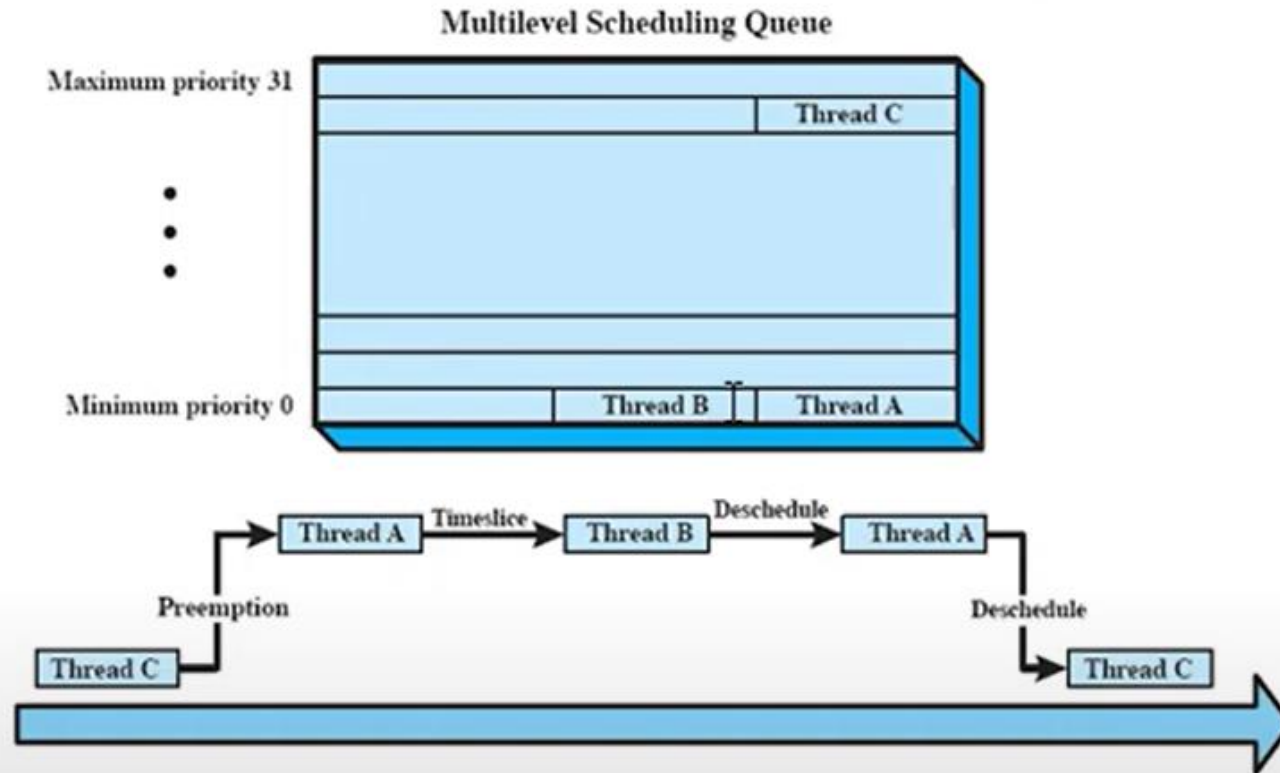
- ▶ Only single thread at each priority level.
- ▶ Pre-emption between different priority levels allowed.
- ▶ Makes the scheduling algorithm simple and hence efficient.

eCos Scheduler

- Bitmap scheduler
 - Efficient for small number of threads active
 - Each thread has different priority
- Multilevel queue scheduler
 - Appropriate when number of threads is dynamic
 - Multiple threads at each priority
 - Time slicing



Multilevel Queue Scheduler Priorities



(b) Multilevel queue scheduler thread operation

Figure 13.7 eCos Scheduler Options

Threads

- ▶ eCos kernel provides API functions for controlling threads within a function.
- ▶ In addition to kernel threads eCos also allows POSIX threads.

Thread handling functions

► Various thread controlling functions exist to

- create and exit threads
- kill or delete threads
- yield a thread.
- delay, suspend and resume threads.
- and more thread specific functions

Synchronization Mechanisms

► The synchronization mechanisms provided by eCos are :

- mutexes
- semaphores
- condition variables
- flags
- message Boxes
- spinlocks (For SMP systems)

Mutexes

- ▶ Mutexes allow multiple threads to share resources serially.
- ▶ Mutexes provide protection against ***Priority Inversion Problem***.
- ▶ eCos provides ***Priority Ceiling Protocol*** and ***Priority Inheritance protocol*** as solutions to above problem.

The Protocols

▶ Priority Ceiling Protocol

- ▶ priority of the owner of mutex is raised to some predefined value.
- ▶ not elegant.

▶ Priority Inheritance

- ▶ priority of owner of thread is raised to highest level of all threads waiting for the mutex.
- ▶ Synchronization calls are costlier.

Mutexes API

► Kernel Mutex controlling API

`cyg_mutex_init()`

`cyg_mutex_destroy()`

`cyg_mutex_lock()`

`cyg_mutex_trylock()`

`cyg_mutex_unlock()`

`cyg_mutex_release()`

`cyg_mutex_set_ceiling()`

`cyg_mutex_set_protocol()`

Semaphores

- ▶ eCos kernel provides API functions for creating and manipulating semaphores.
- ▶ Kernel API is for counting semaphores and not binary semaphores.

Semaphores API

► Kernel Semaphore controlling API

`cyg_semaphore_init()`
`cyg_semaphore_destroy()`
`cyg_semaphore_wait()`
`cyg_semaphore_trywait()`
`cyg_semaphore_timed_wait()`
`cyg_semaphore_post()`
`cyg_semaphore_peek()`

Condition Variables

- ▶ Condition variables are used with mutexes to allow multiple threads to access shared data.
- ▶ eCos kernel provides API to control condition variables.

Condition Variables API

► Kernel API to control condition variables

`cyg_cond_init()`

`cyg_cond_destroy()`

`cyg_cond_wait()`

`cyg_cond_timed_wait()`

`cyg_cond_signal()`

`cyg_cond_broadcast`

Features of eCos RTOS

- ▶ **1. Configurability:** eCos is highly configurable, which means that **developers can select only the components they need for their specific application**. This helps to reduce the size and complexity of the operating system, which is especially important in resource constrained embedded systems.
- ▶ **2. Multitasking:** eCos is a multitasking operating system, which means it can **run multiple tasks or threads simultaneously**. This is essential for real-time systems, where multiple processes must be executed concurrently to ensure timely and reliable operation.
- ▶ **3. Memory Management:** eCos provides advanced memory management features, including **support for virtual memory, memory protection, and dynamic memory allocation**. These features help to ensure the reliability and security of the operating system and the applications running on it.

- ▶ **4. Hardware Abstraction:** eCos provides a hardware abstraction layer (HAL) **that allows developers to write device drivers** and other low-level code that is independent of the underlying hardware. This makes it easier to port eCos to new hardware platforms and reduces the amount of code that must be rewritten for each new device.
- ▶ **5. Real-Time Performance:** eCos is designed to be a real-time operating system, which means that it is optimized for applications **that require precise timing and responsiveness**. It provides features such as interrupt handling, task scheduling, and priority-based preemption to ensure that real-time deadlines are met.
- ▶ **6. Scalability:** eCos is designed to be scalable, which means that it can be used in a **wide range of embedded systems, from small, single-process systems to large, complex systems with multiple processors** and multiple tasks running in parallel.
- ▶ **7. Open Source:** eCos is released under an open-source license, which means that developers can download and modify the source code to meet their specific needs. This has led to a large community of developers contributing to the project and creating new features and applications based on eCos.

Working Principle of eCos RTOS

- ▶ **1. Initialization:** When an embedded system is powered on, the eCos **kernel is loaded into memory and initializes the system hardware, such as setting up interrupt controllers and initializing device drivers.**
- ▶ **2. Thread Scheduling:** eCos provides a priority-based thread scheduler that ensures that **high-priority threads are executed before lower-priority threads.** Each thread is assigned a priority, and the scheduler selects the highest-priority thread that is ready to run.
- ▶ **3. Interrupt Handling:** When an interrupt occurs, the eCos interrupt handler is invoked. **The handler saves the state of the current thread and switches to the interrupt service routine (ISR) for the corresponding interrupt.** When the ISR is finished, the interrupt handler restores the state of the previous thread and returns to the thread scheduler.

- ▶ **4. Memory Management:** eCos provides **memory management services, including virtual memory support, memory protection, and dynamic memory allocation.** Memory pools are used to allocate memory to threads, which reduces the risk of memory fragmentation.
- ▶ **5. Device Drivers:** eCos provides a device driver framework that allows developers to write device drivers for specific hardware platforms. **Device drivers communicate with the hardware through the HAL (Hardware Abstraction Layer), which provides a standardized interface to the hardware.**
- ▶ **6. Interprocess Communication:** eCos provides interprocess communication (IPC) mechanisms that **allow processes to communicate with each other quickly and efficiently.** IPC is based on message passing and supports synchronous and asynchronous messaging, shared memory, and remote procedure calls (RPCs).
- ▶ **7. Configuration:** eCos is highly configurable, and developers can select only the **components they need for their specific application.** The eCos Configuration Tool (ECT) provides a graphical interface that simplifies the configuration process and generates the necessary source code and make files.

Supported Hardware Platforms by eCos RTOS

- ▶ 1. **ARM:** eCos supports a variety of ARM processors, including Cortex-M, Cortex-R, and Cortex-A series.
- ▶ 2. **Intel x86:** eCos supports the Intel x86 architecture, including the Intel Atom and Pentium processors.
- ▶ 3. **MIPS:** eCos supports the MIPS architecture, including the MIPS32 and MIPS64 processors.
- ▶ 4. **PowerPC:** eCos supports the PowerPC architecture, including the PowerPC 400 and PowerPC 500 series processors.
- ▶ 5. **Renesas:** eCos supports a variety of Renesas processors, including the Renesas RZ/A1, RZ/A2, and RZ/G1 series.
- ▶ 6. **Texas Instruments:** eCos supports a variety of Texas Instruments processors, including the Sitara AM335x and AM437x series.
- ▶ 7. **Atmel AVR32:** eCos supports the Atmel AVR32 architecture, including the AVR32 AP7000 and UC3 series processors.
- ▶ 8. **Microchip PIC32:** eCos supports the Microchip PIC32 architecture, including the PIC32MX and PIC32MZ series processors.
- ▶ 9. **Freescale ColdFire:** eCos supports the Freescale ColdFire architecture, including the ColdFire V1, V2, and V3 series processors.